

Practice Questions

Machine Learning (AIML ZG565 / DSE ZG565)

Exam Prep 2025

Contents

1	Module 1: Data Preprocessing & Audit	2
2	Module 2: Regression & Gradient Descent	5
3	Module 3: Logistic Regression	8
4	Module 4: Decision Trees	11
5	Module 5: Model Evaluation	14
6	Module 6: Advanced Scenarios & Emerging Topics	17

1 Module 1: Data Preprocessing & Audit

Q1 (Data Audit - Dec 2025/2023 Pattern). You are tasked with preprocessing a dataset for a Heart Disease prediction model. Examine the raw data sample below. Identify **5 specific data quality issues** and suggest a resolution for each.

ID	Age	Weight	BP	Smoker	Risk
P101	45	70kg	120/80	Yes	High
P102	350	65	130/85	No	Low
P103	Fifty	80kg	140/90	YES	High
P104	35	NaN	110/70	No	?
P105	42	160lbs	120/80	1	Low

Detailed Step-by-Step Solution

1. Logical Outlier / Domain Impossibility (Row P102):

- *Issue:* Age is listed as 350. This is biologically impossible.
- *Resolution:* Do not just keep it. Either check source logs for a typo (likely "35" or "50"), or treat it as a Missing Value and impute it using the median age of the column.

2. Data Type Mismatch / Format Error (Row P103):

- *Issue:* Age is "Fifty" (String) while the column expects Integers. ML models cannot calculate mathematical distance on strings.
- *Resolution:* Parse the string to an integer (50). If parsing fails, treat as missing.

3. Inconsistent Categorical Labels (Column 'Smoker'):

- *Issue:* The column contains mixed representations for the same concept: "Yes", "No", "YES" (case sensitivity), and "1" (numeric boolean).
- *Resolution:* Standardize to a single format. Map {"Yes", "YES", "1"} $\rightarrow$ 1 and {"No"} $\rightarrow$ 0.

4. Unit Inconsistency (Row P105):

- *Issue:* Weight is recorded in "lbs" (160lbs) while others are in "kg" or unlabelled integers (likely kg).
- *Resolution:* Convert to metric. $1 \text{ lb} \approx 0.453 \text{ kg}$.

$$160 \text{ lbs} \times 0.453 = 72.48 \text{ kg}$$

Replace 160 with 72.48 and strip the "kg" suffix from all rows to make the column numeric.

5. Missing Target Variable (Row P104):

- *Issue:* The 'Risk' label is "?".
- *Resolution:* Since this is a Supervised Learning task, we need labeled data for training. You cannot impute the target variable (that is cheating). This row must be **dropped** from the training set.

Q2 (Scaling Logic). You have a dataset with two features: "Annual Income" (Range: 20,000 -

1,000,000) and "Age" (Range: 20 - 60). You plan to use K-Nearest Neighbors (KNN). (a) If you do not scale the data, which feature will dominate the distance calculation? Prove it with a calculation between two points: $A(50k, 30)$ and $B(55k, 50)$. (b) Which scaling technique (Min-Max vs Z-Score) is safer if the Income column has extreme outliers (e.g., a billionaire)?

Detailed Step-by-Step Solution

(a) Dominance Proof: KNN uses distance metrics like Euclidean Distance: $d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. Let's substitute the values:

$$\Delta \text{Income} = 55,000 - 50,000 = 5,000$$

$$\Delta \text{Income}^2 = 25,000,000$$

$$\Delta \text{Age} = 50 - 30 = 20$$

$$\Delta \text{Age}^2 = 400$$

$$\text{Distance} = \sqrt{25,000,000 + 400} \approx \sqrt{25,000,400} \approx 5000.04$$

Conclusion: The contribution of Age (400) is negligible compared to Income (25 million). The distance is practically identical to the difference in Income alone. Income dominates purely due to magnitude, not importance.

(b) Scaling Strategy: Use Z-Score (Standard Scaler). *Reasoning:* Min-Max Scaling maps the range $[Min, Max]$ to $[0, 1]$. If you have a billionaire (10^9) and normal people ($50k$), the denominator ($Max - Min$) becomes huge.

$$x_{scaled} = \frac{50,000 - 0}{1,000,000,000 - 0} = 0.00005$$

All normal data gets squashed into a tiny range near 0, destroying the variance needed to distinguish between them. Z-score ($\frac{x-\mu}{\sigma}$) centers data around 0 and scales by deviation, handling outliers much better without crushing the "normal" data distribution.

Q3 (Sampling Strategy - 2024 Pattern). You have a large dataset of 100,000 transactions, but only 0.1% (100 rows) are labeled as Fraud. You perform a random 80-20 train-test split. (a) What is the risk of a simple random split in this scenario? (b) Name and describe the correct splitting strategy to use.

Detailed Step-by-Step Solution

(a) The Risk: In a purely random split, statistical variance might lead to an uneven distribution of the minority class.

- You might get all 100 fraud cases in the Training set (Test set has 0 fraud cases). Result: You cannot evaluate if the model detects fraud.
- You might get too many fraud cases in the Test set (Training set has very few). Result: The model fails to learn what fraud looks like.

(b) The Solution: Stratified Sampling. This method splits the data such that the *proportion* of classes in the sample matches the proportion in the population.

- *Implementation:* If the original data is 99.9% Non-Fraud and 0.1% Fraud.
- *Train (80%):* Will contain exactly 80 fraud cases and 79,920 non-fraud cases.

- *Test (20%)*: Will contain exactly 20 fraud cases and 19,980 non-fraud cases.

This ensures the model is trained and tested on representative distributions.

Q4 (Feature Selection Scenario). You are predicting "House Price" using a decision tree. Feature A: "House.ID" (Unique for every row). Feature B: "Country" (Value is "USA" for 100% of rows). Decide whether to Keep or Drop these features and justify.

Detailed Step-by-Step Solution

1. **House.ID** → **DROP**. *Reasoning*: This is a High Cardinality feature. Every row has a unique value. A Decision Tree could theoretically split on every single ID (e.g., "If ID is 105, Price is X"). This leads to 100% training accuracy but massive overfitting, as "ID 105" is just a label, not a feature of the house. It has zero generalization power for new houses.
2. **Country** → **DROP**. *Reasoning*: This is a Zero Variance feature. If every house is in "USA", the feature provides zero information gain. It cannot help the model distinguish between an expensive house and a cheap house because the value is identical for both. It adds computational overhead with no benefit.

Q5 (Data Leakage). A student replaces missing values in the *training* set with the mean of the *entire* dataset (Training + Test combined). Explain why this invalidates the test results.

Detailed Step-by-Step Solution

This is a textbook example of **Data Leakage** (specifically, leakage in preprocessing).

- **The Principle**: The Test set simulates "future, unseen" data. You should know absolutely nothing about it during training.
- **The Error**: By calculating the mean of the *entire* dataset, the "mean" value contains information from the Test set rows. When you impute this mean into the Training set, you have essentially injected summaries of the test answers into the study material.
- **Consequence**: The model will perform artificially well on the test set because it has already "seen" the distribution of the test data via the imputed mean. In production (real world), this performance will drop significantly.

Q6 (Encoding). Why is One-Hot Encoding preferred over Label Encoding (0, 1, 2) for nominal categorical features like "Color" (Red, Green, Blue) in Linear Regression?

Detailed Step-by-Step Solution

The Problem with Label Encoding: If you map Red=0, Green=1, Blue=2:

- Linear Regression computes weights based on algebra: $y = w \cdot x$.
- The model interprets the numbers mathematically. It assumes $Blue(2) > Green(1)$, implying Blue is "worth more" than Green.
- It also assumes an order: $|Blue - Green| = 1$ and $|Green - Red| = 1$. It assumes Green is the average of Red and Blue.

For nominal data (colors), no such rank or order exists.

The Solution (One-Hot Encoding): Creates 3 binary columns: Is_Red, Is_Green, Is_Blue.

$$\text{Red} \rightarrow [1, 0, 0], \quad \text{Green} \rightarrow [0, 1, 0]$$

These vectors are orthogonal (perpendicular). There is no "greater than" relationship, allowing the regression model to assign an independent weight to each color.

2 Module 2: Regression & Gradient Descent

Q7 (Manual Gradient Descent - Dec 2025 Pattern). Perform **one iteration** of Batch Gradient Descent for a simple regression model $h_{\theta}(x) = \theta_1 x$ (Assume bias $\theta_0 = 0$ is fixed).

- Data Point: $x = 2, y = 10$.
- Initial Weight: $\theta_1 = 3$. Learning Rate: $\alpha = 0.1$.

Calculate the Prediction, Error, Gradient, and Updated Weight θ_1^{new} .

Detailed Step-by-Step Solution

Step 1: Calculate Prediction ($h_{\theta}(x)$)

$$h_{\theta}(x) = \theta_1 \times x = 3 \times 2 = 6$$

Step 2: Calculate Error (Residual)

$$\text{Error} = h_{\theta}(x) - y = 6 - 10 = -4$$

Step 3: Calculate Gradient ($\frac{\partial J}{\partial \theta_1}$) The cost function derivative for MSE is $(h(x) - y)x$.

$$\text{Gradient} = \text{Error} \times x = -4 \times 2 = -8$$

Step 4: Update Rule The update rule is: $\theta_{new} = \theta_{old} - \alpha \times \text{Gradient}$.

$$\theta_1^{new} = 3 - 0.1 \times (-8)$$

$$\theta_1^{new} = 3 - (-0.8)$$

$$\theta_1^{new} = 3 + 0.8 = \mathbf{3.8}$$

Analysis: The initial prediction (6) was too low (wanted 10). The gradient was negative, so we subtracted a negative (added) to the weight, increasing it from 3 to 3.8. This will make the next prediction higher ($3.8 \times 2 = 7.6$), moving closer to the target.

Q8 (Learning Curve Diagnosis). You train a Polynomial Regression model (Degree 15) on a small dataset.

- Training RMSE: 2.0 (Very Low).
- Validation RMSE: 500.0 (Very High).

(a) Is this High Bias or High Variance? (b) A colleague suggests "Add more features". Is this correct? Why/Why not?

Detailed Step-by-Step Solution**(a) Diagnosis: High Variance (Overfitting).**

- Low Training Error indicates the model has enough capacity to learn the data perfectly.
- High Validation Error indicates it fails to generalize to new data.
- A Degree 15 polynomial is extremely wiggly; it is fitting the random noise in the training set rather than the underlying trend.

(b) Recommendation: Incorrect. Adding *more* features generally increases model complexity.

- We are already suffering from high complexity (Degree 15). Adding features will likely worsen the Variance (make the gap between Train and Validation error larger).
- **Correct Actions:** 1. **Reduce Complexity:** Reduce polynomial degree (e.g., from 15 to 2 or 3). 2. **Regularization:** Add an L2 (Ridge) or L1 (Lasso) penalty to constrain the weights. 3. **More Data:** Get more training examples to smooth out the noise.

Q9 (Regularization Selection). You have a dataset with 10,000 gene features, but you suspect only 5 are actually relevant to the disease. Which Regularization technique (L1 Lasso or L2 Ridge) should you use? Justify using the geometry of the constraint.

Detailed Step-by-Step Solution

Selection: L1 Regularization (Lasso).

Justification:

- **Feature Selection:** Lasso has the unique property of driving the coefficients of irrelevant features to **exactly zero**. This yields a "sparse" model, effectively automatically selecting the subset of relevant features.
- **Geometric Explanation:** The L1 constraint region ($|\theta_1| + |\theta_2| \leq C$) is a **diamond** (or polytope in higher dimensions) with sharp corners on the axes.
- The contours of the Cost Function (ellipses) expand until they touch the constraint region. Statistically, it is highly probable that the ellipses touch the diamond at a **corner** (on an axis) rather than along a flat side.
- Touching a corner implies that one or more coordinates (θ) are 0.
- In contrast, L2 (Ridge) has a circular constraint. The ellipses touch the circle at a tangent point, shrinking values to be *small*, but almost never exactly *zero*.

Q10 (Gradient Descent Pitfall). If your Loss Function increases after every epoch ($J : 100 \rightarrow 150 \rightarrow 500 \dots$), what is the most likely cause? How do you fix it?

Detailed Step-by-Step Solution

Cause: Exploding Gradient (Learning Rate too high).

- The Gradient Descent algorithm attempts to walk "downhill".

- If the Learning Rate α is too large, the step size is huge.
- Instead of stepping into the valley minimum, the algorithm jumps *across* the valley to the other side, landing at a point higher up than where it started.
- In the next step, the gradient is even steeper, causing an even larger jump, leading to divergence (∞).

Fix: Drastically **reduce the Learning Rate** (e.g., if $\alpha = 0.1$, try $\alpha = 0.01$ or $\alpha = 0.001$).

Q11 (Normal Equation vs Gradient Descent). When would you prefer Gradient Descent over the Normal Equation ($\theta = (X^T X)^{-1} X^T y$) for Linear Regression?

Detailed Step-by-Step Solution

You prefer Gradient Descent when **n (number of features) is large** (e.g., $n > 10,000$).

Reasoning (Computational Complexity):

- The Normal Equation requires calculating the inverse of the matrix $X^T X$, which is an $n \times n$ matrix.
- Matrix inversion is an operation of complexity $O(n^3)$.
- If $n = 100,000$, n^3 is massive, and the matrix might not even fit in RAM.
- Gradient Descent is roughly $O(kn^2)$ or even $O(kn)$, making it computationally feasible for very large feature sets.

Q12 (Bias-Variance Tradeoff). What happens to Bias and Variance as you increase the regularization parameter λ in Ridge Regression?

Detailed Step-by-Step Solution

The parameter λ controls the penalty strength.

- **High λ = Strong Penalty:** The model is forced to keep weights θ very small (simple model).
- **Impact on Bias (Increases):** A simpler model is less flexible. It might fail to capture complex patterns in the data (Underfitting).
- **Impact on Variance (Decreases):** A simpler model is less sensitive to noise in the training data. It will look similar regardless of the specific training subset used.

Summary: Increasing λ moves the model from High Variance (Overfitting) toward High Bias (Underfitting).

Q13 (Interpretation). In a Housing Price model (Y in \$), the coefficient for "Distance to City Center (km)" is $\beta = -1500$. Interpret this value.

Detailed Step-by-Step Solution

Interpretation: "Holding all other features constant (*ceteris paribus*), for every **1 kilometer increase** in the distance from the city center, the predicted price of the house **decreases by**

\$1,500.”

The negative sign indicates an inverse relationship: as distance goes up, price goes down.

Q14 (Loss Function). Why is Mean Squared Error (MSE) sensitive to outliers? Suggest a robust alternative.

Detailed Step-by-Step Solution

Sensitivity to Outliers: MSE calculates loss as $(y_{pred} - y_{true})^2$.

- If a typical error is 5, the penalty is $5^2 = 25$.
- If there is one outlier with an error of 100, the penalty is $100^2 = 10,000$.
- The squaring operation disproportionately punishes large errors. In an effort to minimize the sum of errors, the model will shift significantly toward the outlier to reduce that massive 10,000 penalty, often sacrificing accuracy on the normal data points.

Robust Alternative: Mean Absolute Error (MAE): $|y_{pred} - y_{true}|$. It scales linearly. An error of 100 is just 20 times worse than 5, not 400 times worse. **Huber Loss:** Acts like MSE for small errors and MAE for large errors (outliers).

Q15 (Polynomial Features). Can Linear Regression model a non-linear relationship like $y = x^2$? Explain how.

Detailed Step-by-Step Solution

Yes, using **Basis Function Expansion** (or Feature Engineering).

- Linear Regression is linear in *parameters* (θ), not necessarily in input variables (x).
- Model equation: $y = \theta_0 + \theta_1 x + \theta_2 x^2$.
- We create a new feature column x_2 such that $x_2 = (x_{original})^2$.
- To the algorithm, the equation looks like $y = \theta_0 + \theta_1 x_1 + \theta_2 x_2$. This is a standard linear plane in 3D space (y, x_1, x_2) .
- When projected back onto the 2D space (y, x) , it forms a parabolic curve.

3 Module 3: Logistic Regression

Q16 (Numerical Prediction). Logistic Regression Weights: $\theta_0 = -4, \theta_1 = 2$ (Feature x_1). (a) Calculate the probability $P(y = 1)$ for $x_1 = 3$. (b) Calculate the probability for $x_1 = 1$.

Detailed Step-by-Step Solution

Logistic Regression uses the Sigmoid function: $\sigma(z) = \frac{1}{1+e^{-z}}$, where $z = \theta_0 + \theta_1 x$.

(a) For $x_1 = 3$:

$$z = -4 + 2(3) = -4 + 6 = 2$$

$$P(y = 1) = \frac{1}{1 + e^{-2}} \approx \frac{1}{1 + 0.135} \approx \frac{1}{1.135} \approx \mathbf{0.88}$$

(The model is 88% sure this is Class 1).

(b) For $x_1 = 1$:

$$z = -4 + 2(1) = -4 + 2 = -2$$

$$P(y = 1) = \frac{1}{1 + e^{-(-2)}} = \frac{1}{1 + e^2} \approx \frac{1}{1 + 7.389} \approx \frac{1}{8.389} \approx \mathbf{0.12}$$

(The model is only 12% sure this is Class 1).

Q17 (Cost Function Logic). For a single sample with True Label $y = 1$: (a) Model A predicts 0.99. Calculate Log Loss. (b) Model B predicts 0.01. Calculate Log Loss. (c) What does this tell you about the penalty for "confident wrong" answers?

Detailed Step-by-Step Solution

Formula for Log Loss (Single Sample): $Loss = -[y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y})]$. Since $y = 1$, the second term becomes 0. We only look at $-\ln(\hat{y})$.

(a) **Model A (0.99):**

$$Loss = -\ln(0.99) \approx -(-0.01) = \mathbf{0.01}$$

Small penalty because the prediction was close to the truth.

(b) **Model B (0.01):**

$$Loss = -\ln(0.01) \approx -(-4.605) = \mathbf{4.605}$$

Huge penalty.

(c) **Conclusion:** Log Loss applies penalties exponentially.

- Being wrong but uncertain (e.g., predicting 0.4 when true is 1) incurs a moderate penalty.
- Being "**Confidently Wrong**" (predicting 0.01 when true is 1) incurs a massive penalty. This forces the model to fix severe misclassifications quickly during training.

Q18 (Decision Boundary Construction). Weights: $\theta_0 = -10, \theta_1 = 1, \theta_2 = 1$. Derive the equation of the decision boundary line. If $x_1 = 4$, what value of x_2 lies on the boundary?

Detailed Step-by-Step Solution

The decision boundary in Logistic Regression is the line where probability is exactly 0.5. This occurs when $z = \theta^T x = 0$.

Step 1: Set equation to 0

$$\theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0$$

$$-10 + 1(x_1) + 1(x_2) = 0$$

Step 2: Solve for the line equation (x_2 in terms of x_1)

$$x_2 = -x_1 + 10$$

This is a line with slope -1 and intercept 10.

Step 3: Solve for specific point If $x_1 = 4$:

$$x_2 = -4 + 10 = \mathbf{6}$$

So the point (4, 6) lies exactly on the decision boundary.

Q19 (Scaling Impact). Explain why unscaled features can cause the "Vanishing Gradient" problem in Logistic Regression training.

Detailed Step-by-Step Solution

The Mechanism:

1. The derivative of the Sigmoid function $\sigma(z)$ is $\sigma(z) \cdot (1 - \sigma(z))$.
2. This derivative is bell-shaped. It is maximum at $z = 0$ (derivative = 0.25) and approaches **zero** as $z \rightarrow \infty$ or $z \rightarrow -\infty$.
3. If features are unscaled (e.g., $x = 10,000$), the weighted sum $z = \theta x$ will be very large (e.g., 50,000).
4. At $z = 50,000$, $\sigma(z) \approx 1$.
5. The derivative becomes $1 \cdot (1 - 1) = 0$.

The Result: Since the gradient is 0, the weights stop updating. The model "thinks" it has converged, but it is actually just stuck in a saturated region of the activation function. Scaling keeps z in the active region (roughly -5 to $+5$) where gradients are non-zero.

Q20 (Convexity). Why can't we use the Mean Squared Error (MSE) cost function for Logistic Regression?

Detailed Step-by-Step Solution

- **Linear Regression:** $h(x)$ is linear. Plugging a linear function into the square error $(h(x) - y)^2$ results in a quadratic "bowl" shape, which is ****Convex****. Gradient Descent easily slides to the bottom.
- **Logistic Regression:** $h(x) = \frac{1}{1+e^{-z}}$ is non-linear. Plugging this into MSE results in a ****Non-Convex**** function (wavy surface with many hills and valleys).
- **Risk:** Gradient Descent might get stuck in a ****Local Minimum**** (a small valley) and never find the Global Minimum (the optimal weights).
- **Solution:** We use Log Loss (Cross-Entropy), which restores convexity for logistic regression.

Q21 (Multi-Class). How does Logistic Regression handle 3 classes (Cat, Dog, Bird)?

Detailed Step-by-Step Solution

Logistic Regression is natively a binary classifier. To handle multi-class, we use strategies like ****One-vs-Rest (OvR)**** (also called One-vs-All).

1. **Train Model 1:** Cat (Positive) vs [Dog + Bird] (Negative). $\rightarrow$ Get Probability $P(\text{Cat})$.
2. **Train Model 2:** Dog (Positive) vs [Cat + Bird] (Negative). $\rightarrow$ Get Probability $P(\text{Dog})$.
3. **Train Model 3:** Bird (Positive) vs [Cat + Dog] (Negative). $\rightarrow$ Get Probability $P(\text{Bird})$.

Prediction: For a new image, run all 3 models. Choose the class associated with the **highest probability score**.

Q22 (Bias Term). What happens to the decision boundary if we remove the bias term θ_0 from the model?

Detailed Step-by-Step Solution

The decision boundary is defined by $\theta_0 + \theta_1 x_1 + \dots = 0$.

- If we remove θ_0 (set it to 0), the equation becomes $\theta_1 x_1 + \dots = 0$.
- If all input features x are 0, the result is necessarily 0.
- **Geometric Consequence:** The decision boundary is forced to pass exactly through the **origin (0,0)**.
- **Impact:** This is a severe constraint. If the data is separable but the separation line does not pass through the origin (e.g., separating Cluster A at (10,10) and Cluster B at (15,15)), the model will fail to fit the data (High Bias/Underfitting).

4 Module 4: Decision Trees

Q23 (Entropy Calculation - Dec 2025 Pattern). Root Node: [5 Yes, 5 No]. Split Left: [5 Yes, 0 No]. Split Right: [0 Yes, 5 No]. (a) Calculate Root Entropy. (b) Calculate Weighted Child Entropy. (c) Calculate Information Gain.

Detailed Step-by-Step Solution

Formula for Entropy: $H(S) = -p_+ \log_2(p_+) - p_- \log_2(p_-)$.

(a) **Root Entropy:** Total = 10. Yes = 5 ($p_+ = 0.5$), No = 5 ($p_- = 0.5$).

$$H(\text{Root}) = -0.5 \log_2(0.5) - 0.5 \log_2(0.5)$$

Since $\log_2(0.5) = -1$:

$$H(\text{Root}) = -0.5(-1) - 0.5(-1) = 0.5 + 0.5 = \mathbf{1.0}$$

(Maximum impurity).

(b) **Weighted Child Entropy:**

- **Left Node:** [5 Yes, 0 No]. Pure Node. $H(\text{Left}) = 0$.
- **Right Node:** [0 Yes, 5 No]. Pure Node. $H(\text{Right}) = 0$.
- **Weighted Average:**

$$H(\text{Children}) = \frac{5}{10} H(\text{Left}) + \frac{5}{10} H(\text{Right}) = 0.5(0) + 0.5(0) = \mathbf{0}$$

(c) **Information Gain:**

$$\text{Gain} = H(\text{Root}) - H(\text{Children}) = 1.0 - 0 = \mathbf{1.0}$$

This represents a "perfect split" where we gained 1 bit of information (moved from total uncertainty to total certainty).

Q24 (Gini Index). Calculate the Gini Index for a node with [3 Yes, 1 No]. Formula: $Gini = 1 - \sum p_i^2$.

Detailed Step-by-Step Solution

Total samples = 4.

- $P(\text{Yes}) = 3/4 = 0.75$
- $P(\text{No}) = 1/4 = 0.25$

$$\begin{aligned}\text{Sum of Squares} &= (0.75)^2 + (0.25)^2 \\ &= 0.5625 + 0.0625 = 0.625\end{aligned}$$

$$Gini = 1 - 0.625 = \mathbf{0.375}$$

Q25 (Continuous Split). Feature "Age": 10, 20, 30, 40. Target: No, No, Yes, Yes. Explain how the decision tree algorithm finds the split point.

Detailed Step-by-Step Solution

Decision Trees handle continuous variables by testing "thresholds".

1. **Sort Data:** 10(No), 20(No), 30(Yes), 40(Yes).
2. **Identify Change Points:** The target class changes between 20 and 30.
3. **Calculate Midpoints (Candidate Splits):**
 - Between 10 and 20: Split at 15. (Left: 10, Right: 20,30,40). Target split: [1N] vs [1N, 2Y].
 - Between 20 and 30: Split at 25. (Left: 10,20, Right: 30,40). Target split: [2N] vs [2Y].
4. **Evaluate Impurity:**
 - Split at 25 results in a Left Node of [No, No] (Pure) and Right Node of [Yes, Yes] (Pure).
 - This split yields maximum Information Gain.
5. **Result:** The node rule becomes **Age < 25**.

Q26 (Overfitting in Trees). Why does a fully grown Decision Tree typically have High Variance? How do we fix it?

Detailed Step-by-Step Solution

Why High Variance? A fully grown tree splits recursively until every leaf node is "pure" (Entropy = 0).

- If the data contains noise (e.g., one mislabeled outlier), the tree will create a specific, deep branch just to classify that one point correctly.
- This results in an overly complex model that memorizes the training data (Interpolation).
- When shown new data, these specific, deep branches will likely fail (High Variance).

Fixes:

1. **Pruning:** Limit the Max Depth, set a Minimum Samples per Leaf, or prune branches with low Gain.
2. **Ensembling:** Use Random Forests (Bagging) to average out the variance of many trees.

Q27 (Pruning Types). Which is considered "less greedy": Pre-Pruning or Post-Pruning? Why?

Detailed Step-by-Step Solution

Post-Pruning is less greedy and generally better.

- **Pre-Pruning (Early Stopping):** Stops splitting if the immediate Information Gain is below a threshold. It is "greedy" because it looks only one step ahead. It might stop at a split that has zero gain (e.g., XOR problem steps) but leads to a massive gain in the *next* layer.
- **Post-Pruning:** Grows the full tree first (capturing all potential interactions), then works backward from the leaves, removing branches that do not improve Validation Accuracy. It sees the "full picture" before cutting.

Q28 (ID3 Weakness). Why is the ID3 algorithm biased towards attributes like "Patient_ID" with high cardinality?

Detailed Step-by-Step Solution

The Problem: ID3 uses **Information Gain**.

- If we split on "Patient_ID" (where every patient is unique), we create N leaf nodes, each with 1 sample.
- A node with 1 sample is 100% pure (Entropy = 0).
- Therefore, the Information Gain is maximized (Root Entropy - 0 = Max Possible).

The algorithm thinks this is the best feature, but it is actually useless for prediction. **The Fix (C4.5):** Use **Gain Ratio**, which divides the Gain by the "Split Info" (entropy of the split itself). Since splitting into N tiny groups has very high Split Entropy, the Gain Ratio penalizes this attribute heavily.

Q29 (Inductive Bias). Describe the geometric shape of the decision boundary for a Decision Tree.

Detailed Step-by-Step Solution

The boundary consists of **Axis-Parallel Hyper-rectangles**.

- Every split is of the form $x_i > c$.
- This creates a line (2D) or plane (3D) perpendicular to an axis.
- A Decision Tree cannot draw a diagonal line (like $y = x$) or a smooth curve ($y = x^2$) directly.
- It approximates these shapes using a "staircase" pattern (Step functions).

Q30 (Regression Trees). For a Classification Tree, we use Entropy/Gini. What metric is used for splitting in a Regression Tree?

Detailed Step-by-Step Solution

Metric: Variance Reduction (or MSE).

- In regression, we don't have classes. We have continuous numbers.
- The goal is to group data points with similar numerical values.
- For a candidate split, we calculate the Variance (or Standard Deviation) of the target values in the Left Child and Right Child.
- We choose the split that results in the **lowest weighted average Variance**. This ensures predictions (usually the mean of the leaf) are close to the actual values in that leaf.

5 Module 5: Model Evaluation

Q31 (Reverse Engineering - Dec 2025 Pattern). Posterior Probs $P(+)$: [0.9, 0.8, 0.4, 0.6, 0.1]. True Labels: [+ , + , + , - , -]. Threshold $t = 0.5$. Construct the Confusion Matrix.

Detailed Step-by-Step Solution

Step 1: Generate Predictions If $P > 0.5 \rightarrow$ Predict Positive (+). Else Negative (-).

- Sample 1: 0.9 $\rightarrow$ Pred (+)
- Sample 2: 0.8 $\rightarrow$ Pred (+)
- Sample 3: 0.4 $\rightarrow$ Pred (-)
- Sample 4: 0.6 $\rightarrow$ Pred (+)
- Sample 5: 0.1 $\rightarrow$ Pred (-)

Step 2: Compare Predictions vs Truth 1. True (+), Pred (+) $\rightarrow$ **TP** 2. True (+), Pred (+) $\rightarrow$ **TP** 3. True (+), Pred (-) $\rightarrow$ **FN** (Missed Detection) 4. True (-), Pred (+) $\rightarrow$ **FP** (False Alarm) 5. True (-), Pred (-) $\rightarrow$ **TN**

Step 3: Build Matrix

	Pred Positive	Pred Negative
Actual Positive	TP = 2	FN = 1
Actual Negative	FP = 1	TN = 1

Q32 (Metric Selection). In a "Safe Search" filter for children (detecting Adult Content): (a)

What is a False Positive? (b) What is a False Negative? (c) Which metric should you optimize: Precision or Recall?

Detailed Step-by-Step Solution

Let the "Positive" class be "Adult Content" (the thing we want to detect).

- **(a) False Positive:** The model predicts "Adult", but it is actually "Safe". *Result:* A cartoon is blocked. Annoying, but not harmful.
- **(b) False Negative:** The model predicts "Safe", but it is actually "Adult". *Result:* Adult content is shown to a child. This is the worst-case scenario.
- **(c) Optimize Recall.** $\text{Recall} = \frac{TP}{TP+FN}$. We want to minimize False Negatives (FN). High Recall ensures we catch *almost all* adult content, even if we accidentally block a few safe sites (lower Precision) in the process.

Q33 (Accuracy Paradox). You have a dataset with 95% "Class A" and 5% "Class B". Your model has 95% Accuracy. Why might it be useless?

Detailed Step-by-Step Solution

The Paradox: Consider a "lazy" model that simply predicts "Class A" for every single input, ignoring the features entirely.

- It will be correct 95% of the time (Accuracy = 95%).
- However, for Class B (likely the interesting class, e.g., cancer, fraud), it has **0% Recall**.

Accuracy averages the performance. In imbalanced data, the majority class swamps the score. The model is statistically accurate but practically useless for detecting the minority class.

Q34 (ROC Analysis). Model A has $\text{AUC} = 0.5$. Model B has $\text{AUC} = 0.9$. What does Model A represent?

Detailed Step-by-Step Solution

The ROC curve plots True Positive Rate (y-axis) vs False Positive Rate (x-axis).

- The diagonal line $y = x$ represents **Random Guessing** (like flipping a coin).
- The Area Under the Curve (AUC) for this diagonal is 0.5.
- Therefore, Model A ($\text{AUC} = 0.5$) has no predictive power. It is no better than random chance.
- Model B ($\text{AUC} = 0.9$) is excellent (close to 1.0).

Q35 (Macro Avg vs Micro Avg). Class 1 (100 samples): 100% accuracy. Class 2 (10 samples): 0% accuracy. Calculate Macro-Average Accuracy.

Detailed Step-by-Step Solution

Definition: Macro-Average calculates the metric for each class independently, then takes the

unweighted mean of those scores.

1. Accuracy for Class 1: 1.0 (or 100%).

2. Accuracy for Class 2: 0.0 (or 0%).

3. Macro Average:

$$\frac{Acc_{c1} + Acc_{c2}}{2} = \frac{1.0 + 0.0}{2} = \mathbf{0.50} \text{ (or 50\%)}$$

Note: The Micro-Average (global accuracy) would be $\frac{100 \text{ correct}}{110 \text{ total}} = 90.9\%$. The Macro-Average correctly highlights that the model has completely failed on one of the classes.

Q36 (Log Loss vs Accuracy). Model A predicts the correct class with probability 0.51. Model B predicts the correct class with probability 0.99. Compare their Accuracy and Log Loss.

Detailed Step-by-Step Solution

Assuming threshold 0.5:

- **Accuracy:** Both models predict the class correctly (Probability > 0.5). *Accuracy Score:* Model A = 1 (Correct), Model B = 1 (Correct). No difference.
- **Log Loss:** Model A Loss: $-\ln(0.51) \approx 0.67$. Model B Loss: $-\ln(0.99) \approx 0.01$. *Result:* Log Loss rewards Model B heavily for being **more confident**. Accuracy is a discrete step function; Log Loss is continuous and granular.

Q37 (Kappa Statistic). What does a Cohen's Kappa of 0 indicate?

Detailed Step-by-Step Solution

Formula: $\kappa = \frac{p_o - p_e}{1 - p_e}$, where p_o is observed agreement and p_e is expected agreement by chance.

- A Kappa of 0 means $p_o = p_e$.
- **Interpretation:** The model's performance is exactly equivalent to **random chance**. It has learned no meaningful patterns to distinguish classes better than a random guesser would based on class prevalence.

Q38 (Threshold Tuning). If you increase the classification threshold from 0.5 to 0.8, what happens to Precision and Recall?

Detailed Step-by-Step Solution

Raising the threshold makes the model "stricter". It only predicts Positive if it is 80% sure (vs 50% previously).

- **Precision Increases:** Since we only label the "sure bets" as Positive, the proportion of False Positives drops. The subset of predicted positives is cleaner.
- **Recall Decreases:** We will likely miss some "weak" positives (e.g., actual positives that had a probability score of 0.6 or 0.7). These are now classified as Negative (FN), lowering Recall.

6 Module 6: Advanced Scenarios & Emerging Topics

Q39 (Naive Bayes Calculation). Spam (2 msgs): "Buy Now", "Buy". Ham (1 msg): "Hello". Test: "Buy". Calculate $P(\text{Spam}|\text{Test})$ (Unnormalized score).

Detailed Step-by-Step Solution

Naive Bayes Formula: $P(\text{Class}|\text{Doc}) \propto P(\text{Class}) \times \prod P(\text{word}|\text{Class})$.

Step 1: Priors Total Docs = 3. Spam = 2. Ham = 1.

$$P(\text{Spam}) = 2/3$$

$$P(\text{Ham}) = 1/3$$

Step 2: Likelihoods (P(Word="Buy" — Class))

- **Spam Class:** Words are ["Buy", "Now", "Buy"]. Total words = 3. "Buy" appears 2 times.

$$P(\text{"Buy"}|\text{Spam}) = 2/3$$

- **Ham Class:** Words are ["Hello"]. Total words = 1. "Buy" appears 0 times.

$$P(\text{"Buy"}|\text{Ham}) = 0$$

Step 3: Posteriors (Unnormalized)

- **Score(Spam):** $P(\text{Spam}) \times P(\text{"Buy"}|\text{Spam}) = \frac{2}{3} \times \frac{2}{3} = \frac{4}{9}$
- **Score(Ham):** $P(\text{Ham}) \times P(\text{"Buy"}|\text{Ham}) = \frac{1}{3} \times 0 = 0$

Conclusion: The message is classified as **Spam**.

Q40 (Zero Probability). In Q39, if we observe "Hello" in a Spam email during testing, probability becomes 0. How do we fix this?

Detailed Step-by-Step Solution

The Problem: Naive Bayes multiplies probabilities. If a new word (unseen in training class) appears, likelihood is 0. Since $A \times B \times 0 = 0$, the entire probability collapses to zero, regardless of other strong evidence.

The Fix: Laplace Smoothing (Add-1 Smoothing). We add a small count (dummy count) to every word.

$$P(w|c) = \frac{\text{count}(w, c) + 1}{\text{count}(\text{total}, c) + |V|}$$

where $|V|$ is the size of the vocabulary. This ensures that even unseen words have a tiny, non-zero probability (e.g., $\frac{1}{N}$), preventing the multiplication collapse.

Q41 (KNN Distance Metric). You are classifying text documents. Why is Euclidean Distance often a poor choice? What is the alternative?

Detailed Step-by-Step Solution

Why Euclidean Fails: Euclidean distance measures the magnitude (length) of the difference vector.

- Document A: "Science Science" (Vector: [2, 0])
- Document B: "Science" (Vector: [1, 0])
- Document C: "Sports" (Vector: [0, 1])

Euclidean distance might consider Doc A and Doc B "far apart" simply because Doc A is longer (more words), even though they are about the exact same topic.

Alternative: Cosine Similarity. This measures the **angle** between vectors.

- Doc A and Doc B point in the exact same direction (0 degrees). Cosine Similarity = 1.
- It normalizes for document length, focusing on the *content ratio* rather than word count.

Q42 (Curse of Dimensionality). Explain why KNN requires exponentially more data as the number of features increases.

Detailed Step-by-Step Solution

The Logic:

1. KNN relies on finding "local neighbors" nearby.
2. As you add dimensions (features), the volume of the space increases exponentially ($x^1, x^2, x^3 \dots$).
3. In high dimensions, data becomes incredibly ****sparse****. The "space" is mostly empty.
4. To find k neighbors, you have to expand your search radius significantly.
5. Eventually, the distance to the "nearest" neighbor is statistically indistinguishable from the distance to the "farthest" neighbor.
6. To maintain local density, the number of data points required grows exponentially ($10^1 \rightarrow 10^2 \rightarrow 10^3 \dots$) with dimensions.

Q43 (Ensemble Logic). Why does a Random Forest reduce Variance compared to a single Decision Tree?

Detailed Step-by-Step Solution

Mechanism: Bagging (Bootstrap Aggregating).

- A single decision tree is highly sensitive to training data noise (High Variance).
- Random Forest trains N trees on random subsets of data (Bootstrap) and random subsets of features.
- This **decorrelates** the trees. Their errors become independent random variables.
- **Math:** The variance of the average of N independent random variables is:

$$Var(\bar{X}) = \frac{Var(X)}{N}$$

- By averaging the predictions of 100 trees, we drastically reduce the variance without significantly increasing the bias.

Q44 (AdaBoost Weights). In AdaBoost, what happens to the weight of a sample that is misclassified by Tree 1?

Detailed Step-by-Step Solution

Process:

1. AdaBoost trains weak learners sequentially.
2. After Tree 1 trains, we calculate the error.
3. For every sample i that was **misclassified**:

$$w_i^{new} = w_i^{old} \cdot e^{\alpha}$$

(where $\alpha > 0$).

4. The weight of the misclassified sample **Increases**.
5. **Purpose:** When Tree 2 is trained, it sees a weighted dataset. Since the misclassified point now has a huge weight, Tree 2 is forced to focus specifically on getting that point correct to minimize weighted error.

Q45 (Clustering Scenarios). You have data shaped like a "Crescent Moon" or nested circles. (a) Will K-Means cluster it correctly? Why/Why not? (b) Which algorithm should you use?

Detailed Step-by-Step Solution

(a) K-Means will Fail.

- K-Means defines clusters by a central centroid and assumes points belong to the closest centroid (Euclidean distance).
- This assumption creates **linear separation boundaries** (Voronoi tessellations) that result in convex, spherical clusters.
- It cannot wrap around a crescent shape. It will slice the moon in half.

(b) Use DBSCAN (Density-Based Spatial Clustering).

- DBSCAN does not use centroids. It groups points based on "density connectivity".
- As long as there is a chain of dense points connecting one end of the moon to the other, DBSCAN recognizes it as a single cluster, regardless of shape.